

TECH

REAL-TIME UI / WEBSOCKET /

REALTIME / WEBSOCKET

REALTIME / WEBSOCKET

35 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК mdn / общая практика real-time UI

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	WS Q1 · Q2 · Q3 · Q4 · Q5 · Q6	SSE	POLLING	ВЫБОР	WS Q7 · Q8 · Q9 · Q10 · Q11 · Q12	SSE	POLLING	ВЫБОР
ДЕНЬ 2 ТЯЖЁЛЫЙ	BROWSER WS API -FREQ БАТЧИНГ	BACKPRESSURE rAF FLUSH	HIGH	BROWSER WS API -FREQ БАТЧИНГ	BACKPRESSURE rAF FLUSH	HIGH		
ДЕНЬ 3 ТЯЖЁЛЫЙ	DEDUP BACKOFF	ORDER	STALE	RECONNECT	DEDUP BACKOFF	ORDER	STALE	RECONNECT
ДЕНЬ 4 СРЕДНИЙ	DISCONNECT vs STALE T KNOWN VALUE	DEGRADED	LAS	DISCONNECT vs STALE T KNOWN VALUE	DEGRADED	LAS		
ДЕНЬ 5 ТЯЖЁЛЫЙ	MARKET WATCH BOOK NOTIFICATIONS	PRICE TICKER	ORDER	MARKET WATCH BOOK NOTIFICATIONS	PRICE TICKER	ORDER		
ДЕНЬ 6 СРЕДНИЙ	CONSISTENCY ING MOCK	PARTIAL FAILURE MONITORING	TEST	CONSISTENCY ING MOCK	PARTIAL FAILURE MONITORING	TEST		
ДЕНЬ 7 ТЯЖЁЛЫЙ	INGEST ELTA MEMORY	INGEST vs VIEW WORKER	SNAPSHOT+D	INGEST ELTA MEMORY	INGEST vs VIEW WORKER	SNAPSHOT+D		

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ
ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1

НАГРУЗКА

СРЕДНИЙ

6 ВОПРОСОВ

WS

SSE

POLLING

ВЫБОР

ЧТО УЧИМ В ЭТОТ ДЕНЬ

WS

SSE

POLLING

ВЫБОР

ВОПРОСЫ ДНЯ

Q01 Как устроен WebSocket?

Q02 Чем WebSocket отличается от SSE?

Q03 Чем WebSocket отличается от polling?

Q04 Когда выбрать polling?

Q05 Когда выбрать SSE?

Q06 Когда выбрать WebSocket?

Как устроен WebSocket?

ОПИСАНИЕ

Full-duplex протокол поверх TCP с handshake через HTTP Upgrade. После handshake – bidirectional message-passing. Frames типов text/binary/ping/pong/close. Без overhead HTTP-headers.

КАК РАБОТАЕТ

- 01 HTTP Upgrade → ws://.
- 02 Bidirectional frames.
- 03 ping/pong для keepalive.
- 04 No HTTP-overhead per message.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Real-time двусторонний обмен.
- 02 Chat / trading / collab.

ПОДВОДНЫЕ КАМНИ

- 01 Один TCP-connection – load balancer issues.
- 02 Без TLS – no security.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое frame?
- 02 Чем wss от ws отличается?

ПОЛНАЯ СТАТЬЯ

<https://datatracker.ietf.org/doc/html/rfc6455>

ВОПРОС Q02

Чем WebSocket отличается от SSE?

ОПИСАНИЕ

WS – bidirectional binary/text. SSE – server-to-client only, text-only over HTTP. SSE проще (auto-reconnect, EventSource API), но односторонний. WS – для true bidirectional.

КАК РАБОТАЕТ

- 01 WS: bidirectional, binary.
- 02 SSE: server→client, text only.
- 03 SSE: native EventSource auto-reconnect.
- 04 WS: больше overhead настройки.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Bidirectional → WS.
- 02 Server-push only → SSE.

ПОДВОДНЫЕ КАМНИ

- 01 SSE проще, но lim текст.
- 02 WS требует custom reconnect.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое EventSource?
- 02 Когда SSE достаточно?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events

Чем WebSocket отличается от polling?

ОПИСАНИЕ

Polling – клиент N раз/сек делает HTTP-request. WS – постоянное соединение, server push. Polling простой, но overhead и latency. WS эффективен на high-frequency.

КАК РАБОТАЕТ

- 01 Polling: HTTP every N sec.
- 02 WS: persistent connection.
- 03 Polling overhead больше.
- 04 WS lower latency.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Low-frequency updates → polling OK.
- 02 High-frequency → WS.

ПОДВОДНЫЕ КАМНИ

- 01 Polling может miss event между poll.
- 02 WS требует reconnect-логику.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое long-polling?
- 02 Когда polling уместен?

ПОЛНАЯ СТАТЬЯ

<https://www.html5rocks.com/en/tutorials/websockets/basics/>

ВОПРОС Q04

Когда выбрать polling?

ОПИСАНИЕ

Когда: low-frequency обновления (раз в минуту), не требуется push, infrastructure не поддерживает WS (proxies / load balancers), простота важнее.

КАК РАБОТАЕТ

- 01 Updates < 1/min.
- 02 No infra для WS.
- 03 Simple HTTP только.
- 04 Stateless backend.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Background data refresh.
- 02 Status checks.

ПОДВОДНЫЕ КАМНИ

- 01 High frequency → bandwidth waste.
- 02 Latency hit.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое long-polling?
- 02 Чем differ от WS?

ПОЛНАЯ СТАТЬЯ

https://en.wikipedia.org/wiki/Push_technology#Long_polling

Когда выбрать SSE?

ОПИСАНИЕ

Server-to-client only (notifications, live-логи, stock-prices без user-input). Простой setup (EventSource), auto-reconnect, через HTTP (proxy-friendly).

КАК РАБОТАЕТ

- 01 Server push only.
- 02 HTTP-friendly.
- 03 Auto-reconnect.
- 04 Text-only.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Live-logs.
- 02 Notifications.
- 03 Stock-tickers без actions.

ПОДВОДНЫЕ КАМНИ

- 01 No client → server push.
- 02 Text only – binary через base64.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое EventSource?
- 02 Чем SSE от long-polling отличается?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events

ВОПРОС Q06

Когда выбрать WebSocket?

ОПИСАНИЕ

True bidirectional (chat, multiplayer, real-time collab, trading). High-frequency обновления. Binary protocol. Когда custom protocol поверх TCP-like нужен.

КАК РАБОТАЕТ

- 01 Bidirectional.
- 02 High-frequency.
- 03 Binary support.
- 04 Custom protocols.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Chat / messaging.
- 02 Multiplayer games.
- 03 Trading platforms.
- 04 Collaborative editing.

ПОДВОДНЫЕ КАМНИ

- 01 Reconnect-логика своя.
- 02 Load balancer sticky / kubernetes-issues.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое sticky session?
- 02 Чем wss защищён?

ПОЛНАЯ СТАТЬЯ

<https://datatracker.ietf.org/doc/html/rfc6455>

BROWSER WS API**BACKPRESSURE****HIGH-FREQ****БАТЧИНГ**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

BROWSER WS API

BACKPRESSURE

HIGH-FREQ

БАТЧИНГ

rAF FLUSH

ВОПРОСЫ ДНЯ

Q07 Какие проблемы есть у browser WebSocket API?

Q08 Что такое backpressure?

Q09 Как обрабатывать поток 100+ сообщений в секунду?

Q10 Как не делать setState на каждое сообщение?

Q11 Как батчить updates?

Q12 Как использовать requestAnimationFrame для flush?

ВОПРОС Q07

Какие проблемы есть у browser WebSocket API?

ОПИСАНИЕ

Минимальный API – нет встроенного reconnect, heartbeat, queue, subscription-management. Нет HTTP-headers (auth через cookie или query). Binary через ArrayBuffer / Blob.

КАК РАБОТАЕТ

- 01 No built-in reconnect.
- 02 No heartbeat.
- 03 Auth: cookie / query (no headers).
- 04 Binary как ArrayBuffer.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Custom WS-client setups.
- 02 Library design (socket.io, sockette).

ПОДВОДНЫЕ КАМНИ

- 01 Auth в URL = leak.
- 02 Без library – много boilerplate.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое subprotocols?
- 02 Чем socket.io помогает?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>

Что такое backpressure?

ОПИСАНИЕ

Когда producer (server) шлёт быстрее, чем consumer (browser) обрабатывает. Browser-buffer заполняется → memory growth → freeze. Решения: server-side throttling, client signals 'slow down'.

КАК РАБОТАЕТ

- 01 Producer > Consumer rate.
- 02 Browser ws.bufferedAmount растёт.
- 03 Solutions: throttle, drop, signal back.
- 04 Memory growth → freeze.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 High-frequency feeds.
- 02 Bandwidth-limited clients.

ПОДВОДНЫЕ КАМНИ

- 01 Без monitoring bufferedAmount – скрытое.
- 02 Drop без strategy → потеря важного.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое bufferedAmount?
- 02 Чем backpressure отличается от throttle?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/WebSocket/bufferedAmount>

ВОПРОС Q09

Как обрабатывать поток 100+ сообщений в секунду?

ОПИСАНИЕ

Батчинг (накопить 50мс или N сообщений → flush одним setState), dedup по id, виртуализация UI, перенос парсинга в Web Worker, не setState на каждое.

КАК РАБОТАЕТ

- 01 Батч-flush через rAF / setTimeout.
- 02 Dedup by id (Map).
- 03 Virtualization UI.
- 04 Web Worker для parse.
- 05 No setState per message.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading.
- 02 Chat в крупных каналах.
- 03 Multiplayer.

ПОДВОДНЫЕ КАМНИ

- 01 setState per message → render storm.
- 02 Без dedup – duplicate updates.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое rAF-batching?
- 02 Когда Web Worker?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/inp>

Как не делать `setState` на каждое сообщение?

ОПИСАНИЕ

Накапливать в external store (Map / array), flush через interval/rAF одним обновлением. Subscribe компоненты через `useSyncExternalStore` с селектором.

КАК РАБОТАЕТ

- 01 External store + accumulate.
- 02 Flush через rAF / setTimeout.
- 03 `useSyncExternalStore` + selectors.
- 04 Один `setState` на пачку.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 High-frequency UIs.

ПОДВОДНЫЕ КАМНИ

- 01 State в React → render каждый message.
- 02 Без selectors – все consumer ререндерятся.

МОГУТ ДОСПРОСИТЬ

- 01 Чем external store от `useState` отличается?
- 02 Что такое selector?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useSyncExternalStore>

ВОПРОС Q11

Как батчить updates?

ОПИСАНИЕ

Накапливать в Map<id, Message> 50мс или N items, потом flush одним store.set / setState. Map гарантирует уникальность, последний апдейт побеждает (last-write-wins).

КАК РАБОТАЕТ

- 01 Map<id, msg> аккумулятор.
- 02 Timer 50ms или count threshold.
- 03 Flush одним setState.
- 04 Last-write-wins per id.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading-feeds.
- 02 Live-leaderboards.

ПОДВОДНЫЕ КАМНИ

- 01 Большое окно – лаг.
- 02 Маленькое – нет выгоды.

МОГУТ ДОСПРОСИТЬ

- 01 Чем Map от Array выгоднее?
- 02 Какой батч-window?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map

Как использовать requestAnimationFrame для flush?

ОПИСАНИЕ

Накапливать message → при первом – schedule rAF → в callback flush + reset.
Гарантирует один flush per frame, синхронизирован с paint. Альтернатива setTimeout – менее предсказуемо.

КАК РАБОТАЕТ

- 01 onMessage: queue.push + rAF schedule.
- 02 rAF callback: flush + reset.
- 03 One flush per frame.
- 04 Synchronized c paint.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 UI c heavy update visibility.
- 02 Animations + data.

ПОДВОДНЫЕ КАМНИ

- 01 rAF в hidden tab – не fires.
- 02 Без guard – N rAF queued.

МОГУТ ДОСПРОСИТЬ

- 01 Чем rAF от setTimeout(0) отличается?
- 02 Что в hidden tab?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/window/requestAnimationFrame>

ДЕНЬ 3

НАГРУЗКА

ТЯЖЁЛЫЙ

5 ВОПРОСОВ

DEDUP**ORDER****STALE****RECONNECT****BACKOFF**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

DEDUP ORDER STALE RECONNECT BACKOFF

ВОПРОСЫ ДНЯ

- Q13 Как дедуплицировать сообщения?
- Q14 Как обрабатывать out-of-order messages?
- Q15 Как обрабатывать stale packets?
- Q16 Как обрабатывать reconnect?
- Q17 Как проектировать exponential backoff?

Как дедуплицировать сообщения?

ОПИСАНИЕ

Map<id, message> – последний по id побеждает. Set<hash> для exact-duplicates. Версионный счётчик внутри сообщения. Filter at ingest layer перед UI.

КАК РАБОТАЕТ

- 01 Map<id, msg> = dedup last.
- 02 Set<hash> для exact.
- 03 Version-counter в payload.
- 04 Dedup на ingest, не в UI.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading-feeds.
- 02 Chat resends.

ПОДВОДНЫЕ КАМНИ

- 01 Без dedup – duplicate UI updates.
- 02 Wrong dedup-key – потеря updates.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое idempotency-key?
- 02 Чем version-counter полезен?

ПОЛНАЯ СТАТЬЯ

<https://en.wikipedia.org/wiki/Idempotence>

ВОПРОС Q14

Как обрабатывать out-of-order messages?

ОПИСАНИЕ

Sequence-number / timestamp в каждом сообщении. Ингест-слой буферизует, sort, отдаёт в UI в правильном порядке. Drop too-old (older than current state).

КАК РАБОТАЕТ

- 01 seq-number / timestamp.
- 02 Ingest buffer + sort.
- 03 Drop too-old (< current).
- 04 Reorder window.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Distributed pub-sub.
- 02 Multi-server backends.

ПОДВОДНЫЕ КАМНИ

- 01 Без seq – UI 'прыгает'.
- 02 Buffer слишком долгий – latency растёт.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое jitter buffer?
- 02 Чем sequence-number от timestamp отличается?

ПОЛНАЯ СТАТЬЯ

<https://en.wikipedia.org/wiki/Jitter#Mitigation>

Как обрабатывать stale packets?

ОПИСАНИЕ

Сравнивать timestamp с current state. Если `pkt.ts < current.ts` – drop. Server может посылать ts. Client trust source-of-truth (last-write-wins by ts или by version).

КАК РАБОТАЕТ

- 01 Compare `pkt.ts` vs `current.ts`.
- 02 Drop older.
- 03 Server включает ts.
- 04 Last-write-wins by ts/version.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Real-time UIs.
- 02 Cross-server feeds.

ПОДВОДНЫЕ КАМНИ

- 01 Clock skew между server-client.
- 02 Без timestamp – нечего сравнивать.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое NTP?
- 02 Чем server-ts от client-ts надёжнее?

ПОЛНАЯ СТАТЬЯ

<https://datatracker.ietf.org/doc/html/rfc5905>

ВОПРОС Q16

Как обрабатывать reconnect?

ОПИСАНИЕ

При close – wait + reconnect с exponential backoff + jitter. После reconnect – restore subscriptions, replay missed (через server-side cursor / snapshot). Status-event для UI.

КАК РАБОТАЕТ

- 01 onclose → schedule reconnect.
- 02 Backoff: $200\text{ms} \times 2^n + \text{jitter}$.
- 03 Restore subscriptions.
- 04 Server replay since cursor.
- 05 Status events для UI.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой WS-client.

ПОДВОДНЫЕ КАМНИ

- 01 Без backoff → thundering herd.
- 02 Без replay – потеря событий.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое thundering herd?
- 02 Чем jitter защищает?

ПОЛНАЯ СТАТЬЯ

https://en.wikipedia.org/wiki/Exponential_backoff

Как проектировать exponential backoff?

ОПИСАНИЕ

$\text{delay} = \min(\text{base} \times \text{factor}^n + \text{jitter}, \text{maxDelay})$. base 200ms, factor 2, jitter 0-30%, maxDelay 30s. Reset при successful connect. Лимит retries или forever.

КАК РАБОТАЕТ

- 01 $\text{delay} = \min(\text{base} \times 2^n + \text{jitter}, \text{max})$.
- 02 Reset on success.
- 03 jitter 0-30%.
- 04 maxDelay 30s typical.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Reconnect logic.
- 02 API retry.

ПОДВОДНЫЕ КАМНИ

- 01 Без max → задержки растут вечно.
- 02 Без jitter → синхронные retries.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое jitter?
- 02 Чем backoff от throttle отличается?

ПОЛНАЯ СТАТЬЯ

<https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/>

ДЕНЬ 4 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

DISCONNECT vs STALE DEGRADED LAST KNOWN VALUE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

DISCONNECT vs STALE DEGRADED LAST KNOWN VALUE

ВОПРОСЫ ДНЯ

Q18 Как отличать disconnect от stale connection?

Q19 Как показывать degraded real-time state?

Q20 Как хранить last known value?

Как отличать disconnect от stale connection?

ОПИСАНИЕ

Disconnect – onclose сработал. Stale – connection держится, но no messages. Решение: heartbeat (ping/pong каждые N сек). No pong → close + reconnect.

КАК РАБОТАЕТ

- 01 onclose = disconnect detected.
- 02 Stale: ws.open но silent.
- 03 Heartbeat ping/pong.
- 04 No pong N сек → close().

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Network-flaky environments.
- 02 Mobile.

ПОДВОДНЫЕ КАМНИ

- 01 Без heartbeat – silent stale.
- 02 Heartbeat too frequent – overhead.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое keepalive?
- 02 Чем app-level от tcp keepalive отличается?

ПОЛНАЯ СТАТЬЯ

<https://datatracker.ietf.org/doc/html/rfc6455#section-5.5.2>

ВОПРОС Q19

Как показывать degraded real-time state?

ОПИСАНИЕ

Status banner ('connecting / connected / reconnecting / offline'), gray-out stale data, disable trade-actions on degraded, log degradation, metric для alerting.

КАК РАБОТАЕТ

- 01 Status banner.
- 02 Gray-out stale.
- 03 Disable risky actions.
- 04 Degradation events tracking.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading.
- 02 Live-collaboration.

ПОДВОДНЫЕ КАМНИ

- 01 Без banner → user не знает.
- 02 Disabled без explanation → confusion.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое graceful degradation?
- 02 Чем offline-banner полезен?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/offline-cookbook>

Как хранить last known value?

ОПИСАНИЕ

Persist в memory store + optional localStorage / IndexedDB. При reconnect – show last known + indicator stale. При update – overwrite + clear stale flag. Server-side cursor для resume.

КАК РАБОТАЕТ

- 01 In-memory + localStorage.
- 02 On reconnect – show last + stale flag.
- 03 On update – overwrite.
- 04 Cursor for resume.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Real-time UIs.
- 02 Offline-capable.

ПОДВОДНЫЕ КАМНИ

- 01 localStorage quota.
- 02 Stale без UI-indicator → false comfort.

МОГУТ ДОСПРОСИТЬ

- 01 Чем IndexedDB лучше localStorage?
- 02 Что такое resume cursor?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API

ДЕНЬ 5 НАГРУЗКА ТЯЖЁЛЫЙ 4 ВОПРОСА

MARKET WATCH PRICE TICKER ORDER BOOK NOTIFICATION

ЧТО УЧИМ В ЭТОТ ДЕНЬ

MARKET WATCH PRICE TICKER ORDER BOOK NOTIFICATIONS

ВОПРОСЫ ДНЯ

- Q21 Как проектировать market watch UI?
- Q22 Как проектировать price ticker?
- Q23 Как проектировать order book UI?
- Q24 Как проектировать real-time notifications?

Как проектировать market watch UI?

ОПИСАНИЕ

Список инструментов с обновляемой ценой. Виртуализация, batch updates, flash-эффект (composite-only), per-row мемо, sort/filter с deferred, persist user-cols.

КАК РАБОТАЕТ

- 01 TanStack Virtual.
- 02 Batch updates.
- 03 Flash via transform/opacity.
- 04 React.мемо per row.
- 05 useDeferredValue для sort/filter.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading platforms.

ПОДВОДНЫЕ КАМНИ

- 01 Layout-shift при flash → CLS.
- 02 setState per tick → render storm.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое composite-only?
- 02 Когда canvas вместо DOM?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/animations-guide>

ВОПРОС Q22

Как проектировать price ticker?

ОПИСАНИЕ

Бегущая лента с ценами. CSS-animation transform translate, не left/top. Pause-on-hover для UX. Throttle updates. Виртуализация если много инструментов.

КАК РАБОТАЕТ

- 01 transform: translateX animation.
- 02 Pause on hover.
- 03 Throttle updates.
- 04 Virtualization для длинных лент.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Stock-tickers.
- 02 News-tickers.

ПОДВОДНЫЕ КАМНИ

- 01 Animation through left/top → reflow.
- 02 Update per tick → janky.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое composite-only анимация?
- 02 Чем will-change полезен?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/CSS/will-change>

Как проектировать order book UI?

ОПИСАНИЕ

Bids/asks с обновлением 100+/sec. Snapshot+delta WS. Map<price, level>. Виртуализация $\pm N$ rows around mid. Aggregation by tick. Иногда canvas.

КАК РАБОТАЕТ

- 01 Snapshot+delta protocol.
- 02 Map<price, level>.
- 03 Виртуализация around mid-price.
- 04 Aggregation by tick.
- 05 Canvas option.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Crypto / forex L2.

ПОДВОДНЫЕ КАМНИ

- 01 DOM на 1000+ ticks/sec – kill UI.
- 02 Без aggregation – слишком много шума.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое L2 / L3 data?
- 02 Когда canvas?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/canvas-performance>

ВОПРОС Q24

Как проектировать real-time notifications?

ОПИСАНИЕ

WS push, persist в react-query / Zustand, badge count, mark-as-read с optimistic, history через infinite-query, sync между табами через BroadcastChannel.

КАК РАБОТАЕТ

- 01 WS push.
- 02 Optimistic mark-as-read.
- 03 Badge count.
- 04 Infinite-query history.
- 05 BroadcastChannel sync tabs.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 B2C apps.
- 02 SaaS.

ПОДВОДНЫЕ КАМНИ

- 01 Race на mark-as-read.
- 02 Tabs не sync – stale badge.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое BroadcastChannel?
- 02 Чем optimistic полезен?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/BroadcastChannel>

CONSISTENCY**PARTIAL FAILURE****TESTING****MOCK****MONI**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

CONSISTENCY

PARTIAL FAILURE

TESTING

MOCK

MONITORING

ВОПРОСЫ ДНЯ

Q25 Как обеспечить consistency в UI?

Q26 Как обрабатывать partial failure?

Q27 Как тестировать WebSocket logic?

Q28 Как мокать WebSocket в тестах?

Q29 Как мониторить reconnect rate?

Q30 Как мониторить stale data rate?

ВОПРОС Q25

Как обеспечить consistency в UI?

ОПИСАНИЕ

Single source of truth – store. Все компоненты читают через selectors. Updates atomic. Snapshot+delta для resync. Version-counter для conflict resolution.

КАК РАБОТАЕТ

- 01 SSOT – store.
- 02 Atomic updates.
- 03 Selector-based reads.
- 04 Snapshot+delta resync.
- 05 Version per entity.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Real-time UIs.
- 02 Collab tools.

ПОДВОДНЫЕ КАМНИ

- 01 Дубль state → рассинхрон.
- 02 Без atomic → partial states.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое eventual consistency?
- 02 Чем snapshot+delta полезен?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/articles/patterns-of-distributed-systems/eventual-consistency.html>

Как обрабатывать `partial failure`?

ОПИСАНИЕ

Server send только то, что доступно. Client отображает `available`, `indicates missing`.
Retry для `unavailable`. UI per-section `ErrorBoundary`. Fallback values.

КАК РАБОТАЕТ

- 01 Server: `partial-response` (207-like).
- 02 UI: `available` + `indicator`.
- 03 Retry per `missing`.
- 04 `ErrorBoundary` per section.
- 05 Fallback values.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-source dashboards.
- 02 Microservices BE.

ПОДВОДНЫЕ КАМНИ

- 01 Fail entire UI на `partial` – bad UX.
- 02 Без `indication` – user видит 0 как `valid`.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое 207 Multi-Status?
- 02 Чем `partial-response` от `full` отличается?

ПОЛНАЯ СТАТЬЯ

<https://datatracker.ietf.org/doc/html/rfc4918>

ВОПРОС Q27

Как тестировать WebSocket logic?

ОПИСАНИЕ

Mock WS через jest.fn / MSW WS / sinon-websocket. Test reconnect, backoff, dedup, order. Integration test с test-server (Playwright + ws-server fixture).

КАК РАБОТАЕТ

- 01 Mock WS unit tests.
- 02 Integration с ws-server.
- 03 Test reconnect / backoff.
- 04 Test dedup / order.
- 05 Playwright e2e.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 WS-heavy apps.

ПОДВОДНЫЕ КАМНИ

- 01 Без mock – flaky tests.
- 02 E2E против live server – slow.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое MSW WS?
- 02 Чем sinon-websocket полезен?

ПОЛНАЯ СТАТЬЯ

<https://mswjs.io/>

Как мокать WebSocket в тестах?

ОПИСАНИЕ

MSW + ws handlers, или ручной мок (class MockWS implements WebSocket). Контролировать onmessage, simulate close. В Playwright – page.route + custom server.

КАК РАБОТАЕТ

- 01 MSW WS handlers.
- 02 Class MockWS.
- 03 Simulate onmessage / onclose.
- 04 Playwright + test server.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Unit / integration tests.
- 02 E2E reconnect scenarios.

ПОДВОДНЫЕ КАМНИ

- 01 Real WS в unit-тестах = flaky.
- 02 Без timer-mock – async-test slow.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое MSW?
- 02 Чем jest fake timers полезны?

ПОЛНАЯ СТАТЬЯ

<https://mswjs.io/docs/api/ws>

ВОПРОС Q 2 9

Как мониторить reconnect rate?

ОПИСАНИЕ

Counter event 'ws.reconnect'. Histogram time-to-reconnect. Tag: disconnect-reason, env. Alert при rate > threshold. Correlate с backend-инцидентами.

КАК РАБОТАЕТ

- 01 Counter ws.reconnect.
- 02 Histogram reconnect-duration.
- 03 Tag по reason.
- 04 Alert threshold.
- 05 Correlate с BE.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production WS-apps.

ПОДВОДНЫЕ КАМНИ

- 01 Without metric – silent growing.
- 02 No correlation with BE → false attribution.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое RUM?
- 02 Чем correlation полезен?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/rum>

Как мониторить stale data rate?

ОПИСАНИЕ

Track age данных в UI vs threshold. Метрика per-page. Alert при growing rate. Correlate с reconnect / backend-issues.

КАК РАБОТАЕТ

- 01 Track age (now - lastUpdate).
- 02 Threshold per data-type.
- 03 Alert при growing.
- 04 Correlate с reconnect.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Real-time UIs.
- 02 Trading.

ПОДВОДНЫЕ КАМНИ

- 01 Threshold subjective.
- 02 Без correlation – слепо.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое staleness?
- 02 Чем correlation полезен?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/inp>

ДЕНЬ 7

НАГРУЗКА

ТЯЖЁЛЫЙ

5 ВОПРОСОВ

INGEST

INGEST vs VIEW

SNAPSHOT+DELTA

MEMORY

V

ЧТО УЧИМ В ЭТОТ ДЕНЬ

INGEST

INGEST vs VIEW

SNAPSHOT+DELTA

MEMORY

WORKER

ВОПРОСЫ ДНЯ

Q31 Как проектировать ingest layer?

Q32 Как отделить ingest model от view model?

Q33 Как реализовать snapshot + incremental updates?

Q34 Как защититься от memory growth в буфере?

Q35 Когда переносить обработку в Web Worker?

Как проектировать ingest layer?

ОПИСАНИЕ

Отдельный слой между WS и UI: dedup, sort by sequence, drop stale, batch, throttle. Не в React-state. Реализуется через class или vanilla store.

КАК РАБОТАЕТ

- 01 Class IngestLayer / store.
- 02 Dedup by id / hash.
- 03 Sort by sequence.
- 04 Drop stale.
- 05 Batch + throttle output.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 High-frequency feeds.
- 02 Trading / collaborative.

ПОДВОДНЫЕ КАМНИ

- 01 Логика в React-state – медленно.
- 02 Без layer – везде дублируется.

МОГУТ ДОСПРОСИТЬ

- 01 Чем ingest от view layer отличается?
- 02 Что такое pipeline?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/articles/patterns-of-distributed-systems/>

ВОПРОС Q32

Как отделить ingest model от view model?

ОПИСАНИЕ

Ingest – структура для быстрого update (Map<id, raw>). View – derived, оптимизированная для render (Array<sorted>, computed fields). Update view периодически или по event.

КАК РАБОТАЕТ

- 01 Ingest: Map<id, raw>.
- 02 View: Array<sorted>, derived fields.
- 03 Update view via batch.
- 04 Selector-based derive.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading stores.
- 02 Live-leaderboards.

ПОДВОДНЫЕ КАМНИ

- 01 View-update per ingest – performance.
- 02 Inconsistency между ingest/view.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое CQRS-like pattern?
- 02 Чем reselect полезен?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/bliki/CQRS.html>

Как реализовать snapshot + incremental updates?

ОПИСАНИЕ

Initial snapshot – full state, каждое update – delta (added/removed/changed).
Sequence-number в каждом. Resync через snapshot если sequence-gap. Cursor для replay.

КАК РАБОТАЕТ

- 01 Initial snapshot full.
- 02 Delta: { added, removed, changed }.
- 03 Sequence per update.
- 04 Resync on gap.
- 05 Cursor for replay.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Order books.
- 02 Multiplayer state.
- 03 Collaborative editing.

ПОДВОДНЫЕ КАМНИ

- 01 Без sequence – нельзя detect gap.
- 02 Snapshot слишком частый – bandwidth.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое CRDT?
- 02 Чем event-sourcing похож?

ПОЛНАЯ СТАТЬЯ

https://en.wikipedia.org/wiki/Conflict-free_replicated_data_type

ВОПРОС Q34

Как защититься от memory growth в буфере?

ОПИСАНИЕ

Bounded buffer (Map с capacity, drop oldest или sliding window). Cleanup expired entries. Monitor heap. Web Worker для heavy storage.

КАК РАБОТАЕТ

- 01 Bounded Map (LRU eviction).
- 02 Sliding window by timestamp.
- 03 Cleanup interval.
- 04 Monitor heap размер.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Long-running real-time UIs.
- 02 Trading-sessions.

ПОДВОДНЫЕ КАМНИ

- 01 Unbounded buffer → OOM.
- 02 Cleanup too aggressive → loss data.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое LRU cache?
- 02 Чем sliding window полезен?

ПОЛНАЯ СТАТЬЯ

https://en.wikipedia.org/wiki/Cache_replacement_policies#LRU

Когда переносить обработку в Web Worker?

ОПИСАНИЕ

Когда parsing / compute занимает > 50ms (long task). Heavy computations (decode protocol, decompress, calculate aggregates). Worker shares через postMessage / SharedArrayBuffer.

КАК РАБОТАЕТ

- 01 Long task > 50ms.
- 02 Heavy parse / compute.
- 03 postMessage с transferables.
- 04 SharedArrayBuffer для shared memory.
- 05 Web Worker per channel.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading platforms.
- 02 Crypto / decode-heavy.

ПОДВОДНЫЕ КАМНИ

- 01 postMessage overhead.
- 02 Worker no DOM access.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое transferable?
- 02 Чем SharedArrayBuffer полезен?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

ФИНАЛЬНЫЙ ЧЕК - ЛИСТ

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ [] Описать WS / SSE / polling и выбор (Q1-Q6)

- ☐ [] Описать browser WS API, backpressure, high-freq, batching, rAF (Q7-Q12)

- ☐ [] Описать dedup / order / stale / reconnect / backoff (Q13-Q17)

- ☐ [] Описать disconnect vs stale, degraded, last known value (Q18, Q19, Q20)

- ☐ [] Спроектировать market watch / ticker / order book / notifications (Q21-Q24)

- ☐ [] Описать consistency, partial failure, testing, mock, monitoring (Q25-Q30)

- ☐ [] Описать ingest layer, ingest vs view, snapshot+delta, memory, worker (Q31-Q35)

EVENT LOOP / ASYNC МЕТОДИЧКА v1

МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru